UTN
Facultad Regional Córdoba

TUP

Tecnicatura Universitaria en Programación

LABORATORIO DE COMPUTACIÓN I

Unidad Temática N°2:
Sentencias de Manipulación de Datos

Guía Práctica
1° Año – 1° Cuatrimestre

CC BY ND NC

Índice

Introducción 2

Sentencias de Actualización de Datos 3

Problema 1 3
- Sentencia INSERT 4
- Sentencia UPDATE 6
- Sentencia DELETE 9

Problema 2 11

Problema 3 14

Sentencias de Recuperación de Datos 17

Problema 4 17
- a) La lista de selección y ordenación del resultado: SELECT y ORDER BY .. 18
- b) SELECT TOP 18
- c) Test de Comparación 19

Solución del Problema 4 20
- a) SELECT y ORDER BY 20
- b) SELECT TOP 21
- c) Test de Comparación 22

BIBLIOGRAFÍA 23

Introducción

La creación de una base de datos es un proceso complejo que involucra la definición de estructuras de datos, restricciones de integridad y relaciones entre ellas. Este proceso se lleva a cabo a través del uso de lenguajes de definición de datos (DDL).

Una vez que se han creado las estructuras de la base de datos, se puede comenzar a trabajar con los datos reales utilizando el lenguaje de manipulación de datos (DML). El DML permite la inserción, modificación y eliminación de datos de las tablas de la base de datos.

En este documento, nos enfocaremos en la introducción de los conceptos básicos de la DML. Veremos cómo utilizar comandos como INSERT, UPDATE y DELETE para interactuar con los datos de una base de datos. También exploraremos cómo trabajar con cláusulas WHERE para filtrar los datos y cómo utilizar el lenguaje de consulta de datos para recuperar información de la base de datos.

Al final de este documento, tendrás una comprensión básica de cómo trabajar con los datos en una base de datos, lo que te permitirá comenzar a crear aplicaciones que interactúen con los datos de manera efectiva.

Sentencias de Actualización de Datos

Problema 1

En esta ocasión, te presentamos un ejemplo práctico que te permitirá aprender cómo crear una base de datos y trabajar con ella. *Imaginemos que estamos en una institución educativa que necesita llevar un registro de los alumnos que asisten a dicha institución.*

Para ello, deberemos seguir los siguientes pasos:

1. Crear una base de datos llamada **"Registros_Alumnos"** y en ella crear dos tablas con la siguiente estructura: Barrios y Alumnos.

Tabla Barrios:

Campo	Tipo	Restricciones
id_barrio	int	PK, IDENTITY
nombre_barrio	varchar(50)	NOT NULL

Tabla Alumnos:

Campo	Tipo	Restricciones
Legajo	int	PK
nombre	varchar(50)	NOT NULL
apellido	varchar(50)	NOT NULL
calle	varchar(50)	NOT NULL
nro	int	NOT NULL
id_barrio	int	FK Barrios
nro_dni	varchar(20)	NOT NULL
nro_tel	varchar(20)	
e_mail	varchar(50)	
fecha_nac	datetime	

Tabla 1: Elaboración propia.

2. Al crear las tablas, es importante tener en cuenta que la tabla Barrios debe tener como clave principal el campo **id_barrio** con la propiedad **Identity**, mientras que la tabla Alumnos debe tener como clave principal el campo Legajo y como clave foránea el campo **id_barrio**.

Una vez creadas las tablas:

Procederemos a insertar al menos 10 registros en cada una de ellas.

Sentencia INSERT

En SQL Server, INSERT ***es una sentencia que se utiliza para agregar nuevos datos a una tabla existente***. La sintaxis básica de la sentencia INSERT es la siguiente:

```
INSERT INTO nombre_tabla (columna1, columna2, columna3, ...)
                 VALUES (valor1,   valor2,   valor3,   ...);
```

1. Donde "nombre_tabla" es el nombre de la tabla en la que se desea insertar, "columna1, columna2, columna3, ..." son los nombres de las columnas en las que se insertarán los valores y "valor1, valor2, valor3, ..." son los valores que se desean insertar.
2. Tipos de datos: Es importante tener en cuenta los tipos de datos de las columnas de la tabla, ya que deben coincidir con los valores que se están insertando. Si los tipos de datos no coinciden, se pueden producir errores de conversión.
3. Valores nulos: Si una columna permite valores nulos, se puede insertar un valor nulo en esa columna utilizando la palabra clave "NULL" en lugar de un valor.
4. Restricciones de integridad: Si una tabla tiene restricciones de integridad (como claves primarias o claves foráneas), se debe asegurar que los valores que se están insertando cumplan con esas restricciones.

A continuación, nos adentraremos en un análisis detallado de las sentencias, utilizando comandos de SQL. Para ilustrar esto, presentamos el siguiente ejemplo:

- Para la **tabla Barrios**:

```
INSERT INTO barrios (nombre_barrio) VALUES ('CENTRO');
```

Es importante recordar que el campo id_barrio no se incluye en la lista de campos debido a que se ha establecido como Identity. Además, después de cada inserción podemos verificar que los datos han sido guardados correctamente mediante una consulta como la siguiente:

```
SELECT * FROM barrios;
```

- Para la **tabla Alumnos**:

```
INSERT INTO alumnos (legajo, nombre, apellido, calle, nro,
                     id_barrio, nro_dni, nro_tel, e_mail, fecha_nac)
              VALUES (106743, 'Martín', 'Pedraza', 'Los Olmos', 3422,
                      1, '31234567', '' , NULL, '12/03/1988');
```

Tenga en cuenta que:

- Si un campo es de tipo **char, varchar o datetime, se deben ingresar entre comillas simples**.
- En caso de que el campo Legajo no sea **Identity**, se debe incluir en la lista de campos, de lo contrario se producirá un error.
- Todos los campos incluidos en la lista de campos deben tener un valor asignado en la lista de valores. Para los campos numéricos, se puede asignar el valor 0; para los campos de tipo char o varchar, se puede asignar una cadena de longitud cero ('); para los campos nulos, se puede asignar el valor **NULL**.
- Si se incluye un código de área en el número de teléfono, se debe tener en cuenta que la cantidad de dígitos no debe superar la longitud máxima permitida para el campo.

3. Una vez insertados los datos, podemos verificar que se hayan guardado correctamente mediante una consulta como la siguiente:

```
SELECT * FROM alumnos;
SELECT * FROM barrios;
```

Siguiendo estos sencillos pasos, podrás crear una base de datos y trabajar con ella de manera eficiente. Recuerda que estos son solo algunos de los comandos básicos de SQL que podrás utilizar para manipular los datos de tu base de datos.

- **Se solicita Agregue al menos 9 registros adicionales en cada tabla siguiendo los ejemplos anteriores.**

Sentencia UPDATE

Sentencia UPDATE: La sentencia UPDATE ***se utiliza para modificar los registros existentes en una tabla***. La sintaxis básica es la siguiente:

```
UPDATE nombre_de_la_tabla
   SET columna1 = valor1, columna2 = valor2, ...
 WHERE condicion;
```

La cláusula SET indica qué columnas se van a modificar y con qué valores. La cláusula WHERE especifica qué registros se van a actualizar. Es importante tener cuidado al utilizar la sentencia UPDATE, ya que cualquier error puede afectar muchos registros. Algunos consejos para utilizar esta sentencia son:

1. ***Selección de la tabla***: Se debe seleccionar la tabla adecuada para la actualización. Se debe asegurar que la tabla a actualizar tenga los campos y registros adecuados para la actualización.
2. ***Selección de los registros***: Se debe seleccionar los registros que se desean actualizar. Es importante que la consulta de selección sea lo suficientemente específica para evitar actualizar registros no deseados.
3. ***Sintaxis del comando UPDATE***: La sintaxis del comando UPDATE es importante para actualizar los registros correctamente. Se debe especificar el nombre de la tabla y los campos que se van a actualizar, así como las condiciones que deben cumplirse para la actualización.
4. ***Condiciones de actualización***: Es importante especificar las condiciones para la actualización. Las condiciones deben ser lo suficientemente específicas para actualizar solo los registros deseados. Si las condiciones son demasiado amplias, se pueden actualizar registros no deseados.

Veamos algunos ejemplos de la sentencia UPDATE:

- *Ejemplo 1:* ***Agregar el número de teléfono 5945566 al alumno cuyo legajo es 106743.***

  Para esto, se utiliza la sentencia UPDATE de la siguiente manera:

```
UPDATE alumnos
   SET nro_tel = '5945566'
 WHERE legajo = 106743;
```

Es fundamental tener precaución al utilizar la sentencia UPDATE, ya que si no se especifica una condición adecuada puede afectar a múltiples registros de la tabla. En el ejemplo anterior, se utilizó la condición WHERE legajo = 106743 para asegurarse de que se está actualizando únicamente al alumno deseado. ***Es importante destacar que si el campo utilizado para filtrar no es único, la sentencia UPDATE modificará todos los registros que cumplan con la condición especificada.***

El comando SET se utiliza para especificar los cambios que se desean realizar en los campos de la tabla. Se desea modificar el valor del campo **"nro_tel"** del registro seleccionado por la condición WHERE (en este caso, el alumno con legajo 106743) y establecerlo en '5945566'.

- *Ejemplo 2*: ***Agregar una dirección de e-mail al mismo alumno.***

  En este caso, no se especifica cuál es la dirección de **"e_mail"**que se debe agregar, por lo que se tiene libertad para agregar cualquier dirección válida. En este ejemplo, se utilizará como dirección de **"e_mail"** el correo electrónico "ejemplo@correo.com". Para esto, se utiliza la sentencia UPDATE de la siguiente manera:

```
UPDATE alumnos
   SET e_mail = 'ejemplo@correo.com'
 WHERE legajo = 106743;
```

En este caso, la tabla a actualizar es "Alumnos", y el campo a actualizar es **"e_mail"**, cuyo nuevo valor es "ejemplo@correo.com". La condición que se especifica es que el campo **"legajo"** debe tener el valor 106743.

De esta forma, la sentencia UPDATE actualizará únicamente el campo **"e_mail"** del registro cuyo campo "**legajo"** tenga el valor 106743, y dejará intactos los demás registros de la tabla "Alumnos".

- *Ejemplo 3:* ***El mismo alumno del ejercicio anterior cambió su dirección por Caminoti 7689.***

  En este caso, se desea actualizar los datos de la tabla "alumnos" para reflejar el cambio de dirección en los campos **"calle"** y **"nro"**. La nueva dirección es "Caminoti 7689". Para esto, se utiliza la sentencia UPDATE de la siguiente manera:

```
UPDATE alumnos
   SET calle = 'Caminoti', nro = 7689
 WHERE legajo = 106743;
```

En el ejemplo, se están actualizando dos campos de la tabla "Alumnos": **"calle"** y **"nro"**. La sentencia UPDATE indica que se deben actualizar estos campos con los valores "Caminoti" y 7689, respectivamente. En el WHERE se utiliza el filtro del ejercicio anterior ***al alumno con legajo 106743.*** Recuerda verificar la condición WHERE antes de ejecutar la sentencia en la tabla completa.

- *Ejemplo 4**: Supongamos que queremos actualizar el valor del campo "calle" a NULL para todos los registros de la tabla "alumnos" con id_barrio igual a 4.***

  Para esto, se utiliza la sentencia UPDATE de la siguiente manera:

```
UPDATE alumnos
   SET calle = NULL
 WHERE id_barrio = 4;
```

En este caso, el nombre de la tabla es "alumnos" y se especifica que se desea actualizar el campo **"calle"** a **NULL** para ciertos registros, se está indicando que no hay información disponible sobre la calle para esos registros.

La condición que se utiliza es **"id_barrio = 4"**, lo que significa que solo se actualizarán los registros donde el campo **"id_barrio"** tenga el valor 4. Si no hay ningún alumno con "id_barrio" 4, entonces la sentencia UPDATE no modificará ningún valor en la tabla "alumnos".

Es importante destacar que en este ejemplo se utilizó el campo "id_barrio" que pertenece a la tabla "alumnos", y no el nombre del barrio que pertenece a la tabla "barrios". Esto se debe a que la condición de actualización se basa en la relación que existe entre las tablas "alumnos" y "barrios". Y se hace referencia tampoco al campo “nombre_barrio” que pertenece a la tabla "barrios".

En resumen, la sentencia UPDATE solo actualiza aquellos registros que cumplen con la condición especificada en la cláusula WHERE. Si no hay registros que cumplan con la condición, entonces no se modificará ningún valor en la tabla.

Sentencia DELETE

Sentencia DELETE: La sentencia DELETE ***se utiliza para eliminar uno o varios registros de una tabla***. La sintaxis básica es la siguiente:

```
DELETE FROM nombre_de_la_tabla
WHERE condicion;
```

1. ***¡Cuidado con la condición WHERE!*** La cláusula WHERE especifica qué registros deben ser eliminados de la tabla. Por lo tanto, es importante tener cuidado al construir la condición WHERE para asegurarse de que solo se eliminen los registros que se pretenden eliminar. Si la condición WHERE se omite, se eliminarán todos los registros de la tabla.
2. Las sentencias DELETE son permanentes: A diferencia de la sentencia SELECT, que solo recupera datos de la base de datos, la sentencia DELETE elimina datos permanentemente. Por lo tanto, es importante asegurarse de que los registros que se van a eliminar sean los correctos.
3. Puede haber consecuencias en otras tablas: Si la tabla que se va a eliminar tiene claves externas, es decir, campos que hacen referencia a otra tabla, es importante tener en cuenta las consecuencias que la eliminación de los registros de la tabla pueden tener en otras tablas relacionadas.

Veamos algunos ejemplos de la sentencia DELETE:

- *Ejemplo 1:* ***Eliminar un alumno en particular por su número de legajo****.*

Para hacerlo, podemos utilizar la siguiente consulta:

```
DELETE FROM alumnos
WHERE legajo = 106743;
```

Digamos que queremos eliminar un alumno en particular de la tabla Alumnos por su número de legajo. Supongamos que el legajo del alumno que deseamos eliminar es 106743. Esta consulta eliminará el registro correspondiente al alumno con el número de legajo especificado (106743) de la tabla Alumnos.

- *Ejemplo 2:* ***Supongamos que deseamos eliminar todos los alumnos que viven en el barrio con id_barrio igual a 10.***

Supongamos que deseamos eliminar todos los alumnos que viven en el barrio con id_barrio igual a 10. Para hacerlo, podemos utilizar la siguiente consulta:

```
DELETE FROM Alumnos
WHERE id_barrio = 10;
```

Esta consulta eliminará todos los registros de la tabla Alumnos que coincidan con la condición especificada (id_barrio = 10), es decir, todos los alumnos que viven en el barrio con ese identificador.

- *Ejemplo 2:* ***Eliminar todos los alumnos que nacieron antes de 2003****.*

  Supongamos que deseamos eliminar todos los registros de la tabla Alumnos que corresponden a alumnos que nacieron antes de 2003. Para hacerlo, podemos utilizar la siguiente consulta:

```
DELETE FROM Alumnos
WHERE fecha_nac < '01/01/2003';
```

Esta consulta eliminará todos los registros de la tabla Alumnos que tengan una fecha de nacimiento anterior al 1 de enero de 2003, es decir, todos los alumnos que nacieron antes de esa fecha.

Es importante tener en cuenta que la fecha debe especificarse en el formato 'DD-MM-YYYY' para que la consulta funcione correctamente. Además, es posible ajustar la fecha a cualquier otro valor para eliminar registros basados en diferentes criterios de fecha, como antes o después de una fecha determinada, o entre dos fechas específicas.

Problema 2

Crear una base de datos para una compañía de servicios de televisión por cable que consta de dos tablas: Abonados y Planes. La tabla Abonados contiene información personal, como nombre, apellido, dirección, teléfono y correo electrónico, así como la fecha en que se dieron de alta. La tabla Planes contiene información sobre los planes de servicio ofrecidos por la compañía, incluyendo el nombre del plan, una descripción y el precio mensual correspondiente. Estas dos tablas se relacionan entre sí mediante una relación uno a muchos, lo que significa que cada abonado puede estar asociado a un solo plan, mientras que un plan puede estar asociado con muchos abonados

A continuación, se presenta la estructura de la Tabla Abonados y la Tabla Planes.

Tabla Abonados:

Campo	Tipo	Restricciones
id_abonado	int	**PK, IDENTITY**
nombre	varchar(50)	NOT NULL
apellido	varchar(50)	NOT NULL
direccion	varchar(255)	
telefono	varchar(20)	
e_mail	varchar(50)	NOT NULL
fecha_alta	datetime	NOT NULL
id_plan	int	**FK Planes NOT NULL**

Tabla Planes:

Campo	Tipo	Restricciones
id_plan	int	**PK, IDENTITY**
nombre_plan	varchar(50)	NOT NULL
descripcion	varchar(255)	
precio_mensual	decimal(10,2)	NOT NULL

Tabla 2: Elaboración propia.

Se solicita:

1. Crea la base de datos para la compañía de servicios de televisión por cable.
2. Crea las tablas "Abonados" y "Planes" en la base de datos. Es importante recordar que antes de comenzar cualquier ejercicio, ***uno se debe preguntar cuál tabla debe ser creada primero***.

   En este caso, se debe crear primero la tabla **"Abonados"**, ya que la tabla **"Planes"** tiene una relación de clave foránea con la tabla "**Abonados"**. Al crear primero la tabla **"Abonados"**, se asegura que la clave primaria de la tabla esté disponible para ser utilizada como clave foránea en la tabla **"Planes"**.

INSERT

Una pregunta relevante para reflexionar es la siguiente: si el campo **"id_plan"** de la tabla **"Abonados"** tiene la restricción **"NOT NULL"**, *¿es obligatorio insertar un valor en ese campo?* En este caso, *¿qué tabla debería ser insertada primero, la tabla "Abonados" o la tabla "Planes"?*

3. Agrega 5 registros diferentes en la tabla Planes, cada uno con valores únicos para los campos de nombre del plan, descripción y precio mensual correspondiente.
4. Inserta 5 registros diferentes en la tabla Abonados, cada uno con valores únicos para los campos de nombre, apellido, dirección, teléfono, correo electrónico y fecha de alta.
5. Agrega un nuevo registro en la tabla Planes con nombre "Plan Básico", una breve descripción y un precio mensual de 29.99.
6. Inserta un nuevo registro en la tabla Abonados con nombre "Ana Pérez", dirección "Calle 123", teléfono "555-1234" y correo electrónico "ana@example.com", junto con un plan que se haya creado previamente.
7. Inserta un nuevo registro en la tabla Abonados con nombre "Juan Gómez", dirección "Avenida Principal 567", teléfono "555-5678" y correo electrónico "juan@example.com", con fecha de alta en el momento actual y asociado al plan con nombre "Plan Básico".

UPDATE

8. Actualiza el precio mensual del plan con id_plan = 3 a 49.99.
9. Actualiza el correo electrónico del abonado con id_abonado = 10 a "nuevo_correo@example.com".
10. Actualiza la descripción del plan con id_plan = 4 a "Plan avanzado con más beneficios".
11. Actualiza la fecha de alta del abonado con id_abonado = 7 a la fecha 15-02-2022.
12. Actualiza el nombre del plan con id_plan = 2 a "Plan Premium Plus".
13. Actualiza la dirección del abonado con id_abonado = 1 a "Calle 456, Ciudad de Córdoba".

14. Actualiza el campo "descripcion" y "precio_mensual" del registro con id_plan = 1 en la tabla Planes. La nueva descripción es "Plan Estándar" y el nuevo precio mensual es de 49.99.
15. Actualiza los campos "nombre_plan", "descripcion" y "precio_mensual" del registro con id_plan = 5 en la tabla Planes. El nuevo nombre del plan es "Plan Premium", la nueva descripción es "El plan más completo con acceso a todas las funcionalidades" y el nuevo precio mensual es de 99.99.
16. Actualiza el campo "descripcion" en la tabla Planes, estableciendo la cadena "Oferta especial" para todos los registros cuyo precio mensual sea superior a 50.00.

DELETE

17. Elimina el registro de la tabla Abonados que tenga el id_abonado igual a 10.
18. Borra todos los registros de la tabla Planes que tengan un precio_mensual menor o igual a 10.00.
19. Elimina todos los registros de la tabla Abonados que se hayan dado de alta antes del año 2020.
20. Elimina todos los registros de la tabla Abonados que estén asociados al plan con id_plan igual a 5.

Problema 3

Una librería mayorista cuenta con una base de datos llamada ***LIBRERIA*** que tiene la siguiente estructura:

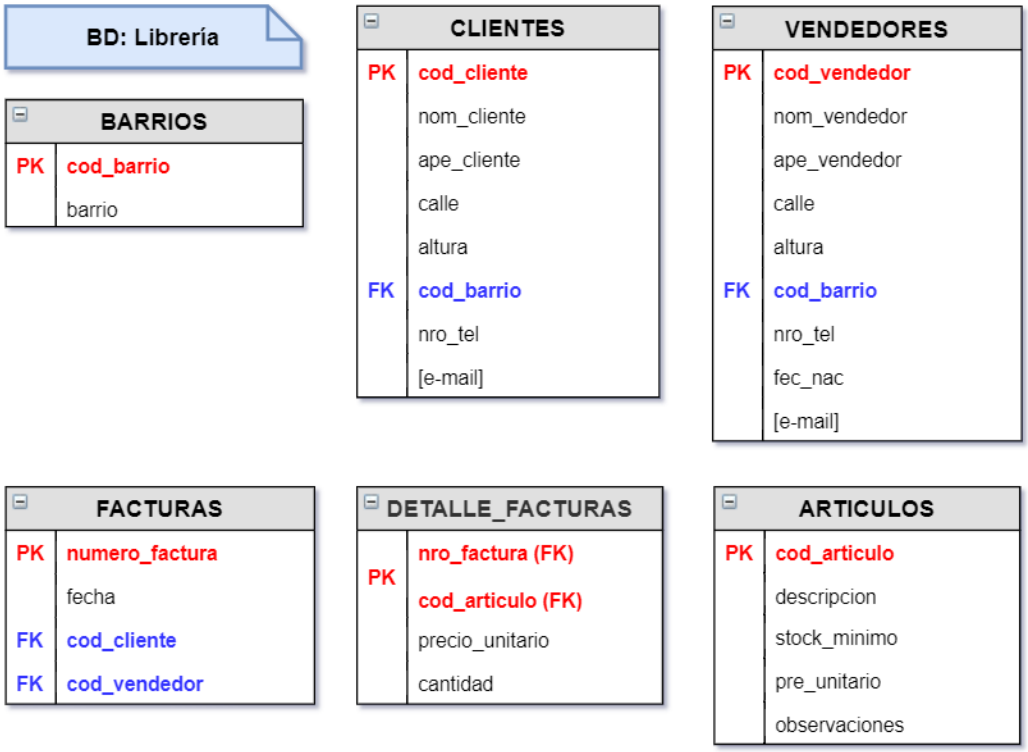

Ilustración 1: Elaboración propia

El encargado de la librería necesita realizar una serie de actualizaciones.

1. Ingresar los artículos que están en la siguiente lista:

Descripción del articulo	Stock	Precio unitario	Observaciones
Lápices Color largos * 12 u. Bic	**50**	**101,5**	
Conjunto Geométrico Maped	150	20,5	Regla, escuadra y transportador
Repuesto Gloria rayado	120	326,3	200 hojas
Repuesto Rivadavia	85	465,9	260 hojas, margen reforzado

Tabla 3: Elaboración propia.

Para insertar nuevas filas en una tabla, se utiliza la sentencia **INSERT**. Por ejemplo, si queremos agregar el primer artículo de la lista anterior a la tabla de artículos, debemos escribir la siguiente sentencia:

```
INSERT INTO articulos (descripcion, stock_minimo, pre_unitario)
                VALUES ('Lápices Color largos * 12 u. Bic ', 50, 101.5);
SELECT * FROM articulos;
```

Es importante tener en cuenta lo siguiente:

- El campo cod_articulo no se incluye en la lista de campos porque es una clave principal de tipo Identity, al igual que cod_vendedor y cod_cliente.
- Si un campo clave principal no es de tipo Identity, debe incluirse en la lista de campos para evitar errores.
- Los campos de tipo char, varchar y datetime se ingresan entre comillas simples.
- Todos los campos incluidos en la lista de campos deben tener un valor en la lista de valores o bien 0 (cero) para numéricos, " (cadena de longitud cero) para char o varchar, o bien NULL.
- Si el número de teléfono es de tipo int y se incluye el código de área, la cantidad de dígitos no será suficiente.
- Si el valor que se intenta agregar a una de las columnas no cumple con alguno de los constraints establecidos, la operación se abortará inmediatamente.

2. La librería requiere la incorporación de los siguientes vendedores:

Apellido	Nombre	Calle	Altura	Tel.	Mail	Fec. Nac.	Barrio
Monti	Juan	Altoaguirre	1245	4522122			5
Sena	Rosa	Av. Velez Sarsfield	25		rsena@hotmail.com	24973	1

Tabla 4: Elaboración propia.

- Ahora ustedes deberán agregar tres clientes.

3. Algunos errores fueron encontrados días después en la carga de datos y será necesario corregirlos. Se solicitó modificar el artículo cuya descripción es "Conjunto Geométrico" por "Conjunto Geométrico de Plástico". En primer lugar, se verificará el cod_articulo correspondiente.

   Si se han seguido las instrucciones de inserción de datos proporcionadas previamente, el código de artículo solicitado sería el 29. A continuación, se muestra cómo realizar la actualización:

```
UPDATE articulos
   SET descripcion = 'Conjunto geométrico de plástico'
 WHERE cod_articulo = 29;
```

En lugar de buscar dentro del listado para encontrar un elemento específico, ***¿existe algún otro tipo de test de búsqueda que se pueda utilizar?*** Es posible que buscar dentro del listado no sea la forma más eficiente o efectiva de encontrar lo que se necesita. Por lo tanto, es importante considerar otras opciones de búsqueda que puedan ser más apropiadas para el caso en cuestión (pista es que ya lo veremos otras formas de búsqueda más adelante en la materia).

Se verifica si se logró realizar la actualización correctamente:

```
SELECT * FROM articulos WHERE cod_articulo = 29;
```

Luego se nos pidieron las siguientes modificaciones:

a) Para el artículo cuya descripción es “Lápices Color largos * 12 u” Bic; cambie el stock mínimo por 100, las observaciones por “Caja con motivos de Disney” y al precio por 17.20.

b) Para el artículo cuya descripción es “Repuesto Rivadavia”, cambie la descripción por “Repuesto Rivadavia cuadriculado” y las observaciones por “48 hojas”.

c) Al vendedor Monti Juan cambiar la fecha por 10/10/1970 y agregarle una dirección de e-mail.

d) Al vendedor Sena Rosa, cambiar el teléfono por 4522221 y la dirección por Av. Vélez Sarsfield 125 - Centro.

e) Aumentar el precio unitario en un 15% pero de aquellos artículos cuyo precio unitario sea inferior a 20.

f) Aumentar el precio unitario en un 10% pero de aquellos artículos cuyo precio unitario esté entre 20 y 30.

Tener en cuenta que, si la actualización de una fila no cumple con una restricción o regla, infringe la configuración de valores NULL o si el nuevo valor es de un tipo de datos incompatible, se cancela la instrucción, se devuelve un error y no se actualiza ningún registro.

4. Por errores en la carga de datos de algunos registros habrá que eliminarlos y para ello utilizaremos la sentencia Delete cuya sintaxis es:

```
DELETE Nombre_tabla
[WHERE Condición]
```

- *Tener en cuenta que si no se especifica la cláusula WHERE,* ***se borran todas las filas de una tabla.***

Los registros para borrar son los siguientes:

a) Al vendedor cuyo apellido y nombre es “Sena Rosa”.

b) El artículo cuya descripción es “Repuesto Gloria rayado”.

c) Elimine dos registros de los que Ud. haya ingresado en los puntos anteriores.

Sentencias de Recuperación de Datos

Problema 4

El vivero de la ciudad posee una base de datos cuyo diagrama se presenta a continuación. El administrador del vivero requiere obtener información precisa y actualizada para tomar decisiones informadas en su trabajo diario.

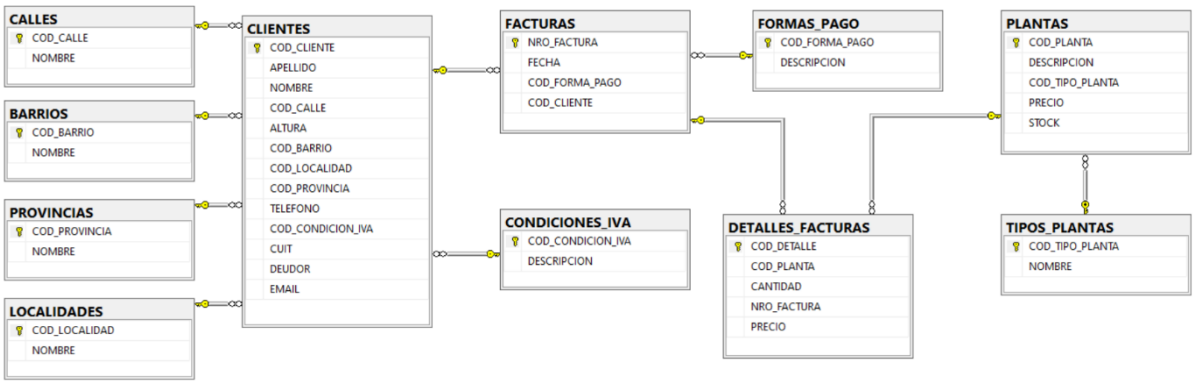

Ilustración 2: Elaboración propia.

Para ello abra la base de datos VIVERO_FENIX y resuelva las consultas que se piden a continuación.

a) La lista de selección y ordenación del resultado: SELECT y ORDER BY

1. Obtener un listado de los clientes de la empresa, mostrando únicamente el código, apellido y nombre, ordenado por código. Rotular los campos como Código de Cliente, Apellido y Nombre.
2. Obtener un listado de los clientes de la empresa, mostrando únicamente el código, apellido y nombre, ordenado por apellido y nombre en forma ascendente. Rotular los campos como Código de Cliente, Apellido y Nombre.
3. Obtener un listado de las plantas que se venden, mostrando el código, descripción y precio, ordenadas por código en forma descendente.
4. Obtener el stock de cada planta, ordenado por la descripción de las plantas. Rotular los campos como Nombre Planta y Stock Actual.
5. Obtener un listado de las localidades que compran a la empresa, ordenadas alfabéticamente.
6. Obtener el apellido y nombre del cliente en una sola columna, además de su código.
7. Obtener todos los datos de las plantas y calcular el precio con un descuento del 10%.
8. Obtener el precio total de stock de cada planta.

b) SELECT TOP

9. Necesito solo un listado de algunos 5 clientes sin deuda (deudor='N'), mostrando únicamente el código, apellido y nombre, teléfono y email. Rotular.
10. Se requiere un listado de las últimas 6 facturas cargadas desde el 11/09/2020 en la base de datos.
11. Listado de las 3 plantas más caras con un stock mayor a 50.

c) Test de Comparación

12. Obtener un listado de todos los clientes deudores (deudor='S'), mostrando únicamente el código, nombre y apellido.
13. Obtener un listado de todas las plantas cuyo stock sea mayor a 20, ordenado por stock.
14. Obtener un listado de todas las plantas cuyo stock no sea 30, ordenado por código.
15. Obtener un listado de todas las facturas cuyas fechas sean posteriores al 01/06/09.
16. Obtener un listado de todas las plantas cuyo stock es menor a 10 unidades.

Solución del Problema 4

a) SELECT y ORDER BY

1.
```
SELECT COD_CLIENTE 'Código de Cliente',
       APELLIDO 'Apellido',
       NOMBRE 'Nombre'
  FROM CLIENTES
 ORDER BY COD_CLIENTE ASC;
```
2.
```
SELECT COD_CLIENTE 'Código de Cliente',
       APELLIDO 'Apellido',
       NOMBRE 'Nombre'
  FROM CLIENTES
 ORDER BY APELLIDO ASC, NOMBRE ASC;
```
3.
```
SELECT COD_PLANTA 'Código',
       DESCRIPCION 'Descripción',
       PRECIO 'Precio'
  FROM PLANTAS
 ORDER BY COD_PLANTA DESC;
```
4.
```
SELECT DESCRIPCION 'Nombre Planta',
       STOCK 'Stock Actual'
  FROM PLANTAS
 ORDER BY DESCRIPCION ASC;
```
5.
```
SELECT NOMBRE 'Localidad'
  FROM LOCALIDADES
 ORDER BY NOMBRE ASC;
```
6.
```
SELECT COD_CLIENTE 'Código de Cliente',
       APELLIDO + ', ' + NOMBRE 'Apellido y Nombre'
  FROM CLIENTES;
```

Si bien en este ejercicio no se especifica el ordenamiento de las columnas, una buena practica es siempre agregar un ordenamiento.

```
SELECT COD_CLIENTE 'Código de Cliente',
       APELLIDO + ', ' + NOMBRE 'Apellido y Nombre'
  FROM CLIENTES
 ORDER BY 1;
```

7.
```
SELECT DESCRIPCION 'Nombre Planta',
       PRECIO 'Precio Actual',
       PRECIO * 0.9 'Precio con descuento'
  FROM PLANTAS
 ORDER BY DESCRIPCION;
```

Este ejercicio puede ser abordado de diferentes maneras, y la elección de una u otra dependerá del método utilizado para calcular el porcentaje.

```
SELECT DESCRIPCION 'Nombre Planta',
       PRECIO 'Precio Actual',
       PRECIO - PRECIO * 0.1 'Precio con descuento'
  FROM PLANTAS
 ORDER BY DESCRIPCION, 2 DESC, 3 DESC;
```

8.
```
SELECT DESCRIPCION 'Nombre Planta',
       STOCK * PRECIO 'Precio total de stock'
  FROM PLANTAS
 ORDER BY 1, 2 DESC;
```

b) SELECT TOP

9.
```
SELECT TOP 5 COD_CLIENTE 'Código de Cliente',
             APELLIDO + ', ' + NOMBRE 'Apellido y Nombre',
             TELEFONO,
             EMAIL
  FROM CLIENTES
 WHERE DEUDOR = 'N';
```

10.
```
SELECT TOP 6 NRO_FACTURA, FECHA, COD_FORMA_PAGO, COD_CLIENTE
  FROM FACTURAS
 WHERE FECHA >= '11/09/2020'
 ORDER BY FECHA DESC; -- OJO CON EL ORDEN
```

11.
```
SELECT TOP 3 COD_PLANTA, DESCRIPCION, PRECIO, STOCK
  FROM PLANTAS
 WHERE STOCK > 50
 ORDER BY PRECIO DESC; -- OJO CON EL ORDEN
```

c) Test de Comparación

12.

```
SELECT COD_CLIENTE, NOMBRE, APELLIDO
  FROM CLIENTES
WHERE DEUDOR = 'S';
```

13.

```
SELECT COD_PLANTA, DESCRIPCION, PRECIO, STOCK
  FROM PLANTAS
 WHERE STOCK > 20
 ORDER BY STOCK;
```

14.

```
SELECT COD_PLANTA, DESCRIPCION, PRECIO, STOCK
  FROM PLANTAS
 WHERE STOCK <> 30
 ORDER BY COD_PLANTA;
```

15.

```
SELECT NRO_FACTURA, FECHA, COD_FORMA_PAGO, COD_CLIENTE
  FROM FACTURAS
 WHERE FECHA > '01/06/2009'
  ORDER BY FECHA;
```

16.

```
SELECT COD_PLANTA, DESCRIPCION, PRECIO, STOCK
  FROM PLANTAS
 WHERE STOCK < 10
 ORDER BY COD_PLANTA;
```

BIBLIOGRAFÍA

- Gorman K., Hirt A., Noderer D., Rowland-Jones J., Sirpal A., Ryan D. & Woody B (2019) Introducing Microsoft SQL Server 2019. Reliability, scalability, and security both on premises and in the cloud. Packt Publishing Ltd. Birmingham UK
- Microsoft. SQL Server 2016. Disponible en: https://www.microsoft.com/es-es/sql-server/sql-server-2016
- Opel, A. & Sheldon, R. (2010). Fundamentos de SQL. Madrid. Editorial Mc Graw Hill
- Varga S., Cherry D., D'Antoni J. (2016). Introducing Microsoft SQL Server 2016 Mission-Critical Applications, Deeper Insights, Hyperscale Cloud. Washington. Microsoft Press

Atribución-No Comercial-Sin Derivadas

Se permite descargar esta obra y compartirla, siempre y cuando no sea modificado y/o alterado su contenido, ni se comercialice. Referenciarlo de la siguiente manera: Universidad Tecnológica Nacional Facultad Regional Córdoba (S/D). Material para la Tecnicatura Universitaria en Programación, modalidad virtual, Córdoba, Argentina.